Loop

TN

Loop is used to perform a set of instructions repeatedly. It involves repeating some portion of the program either a specified number of times or until a particular condition is being satisfied. This repetitive operation is done through a loop control instruction.

A looping process includes the following four steps:-

1) Initialization of a condition variable.
2) Execution of the statements in the loop.
3) Test for a specified value of the condition variable for execution of the loop.
4) Incrementing or updating the condition variable.

In C, there are 3 types of loops:-

1) for loop
2) while loop
3) do-while loop

1) for loop

The general form of the for loop is :-

```
for(initialization; test-condition; increment/decrement)
{
    body of the loop
}
```

The execution of the for statement is as follows:-

i) Initialization of the control variables is done first, using assignment statements such as i=1. Here, the variable i is called loop-control variable.

ii) The value of the loop variable is tested using

Tanumoy Nag

Scanned by CamScanner

the test condition. The test-condition is a relational expression, such as $i < 10$ that determines when the loop will exit.

If the condition is true, the body of the loop is ~~exet~~ executed, otherwise the loop is terminated and the execution continues with the statement that immediately follows the loop.

iii) After evaluating the last statement in the loop, control is transferred back to the for statement. Next, the control variable is incremented using ~~an or~~ the statement such as $i = i + 1$. ~~The control~~ The new value of the control variable is again tested to see whether it satisfies the loop condition or not. If the condition is satisfied, the body of the loop is again executed. This process continues till the value of the control variable fails to satisfy the test condition.

Eg. to display all integers from 1 to 10, the following for loop can be used:-

```
for (i=1; i<=10; i++)
    printf("%d", i);
```

initialization, condition, increment

2) <u>while loop</u>:-

The general form of the while loop is:-

```
while (test condition)
{
    body of the loop
}
```

<u>while</u> is an <u>entry-controlled</u> loop statement. The test-condition is evaluated and if the condition is true, then the body of the loop is

Tanumoy Nag

Scanned by CamScanner

executed. After execution of the body of the loop, the test-condition is once again evaluated and if it is true, the body is executed once again. This process continues until the test-condition becomes false and the control is transferred out of the loop.

Eg. to display all integers from 1 to 10, ~~the~~ using while loop:-

```
i=1;    // Initialization
while (i<=10) // test-condition
{
    printf("%d", i);
    i++;   // increment
}
```

The variable i is called the loop-control variable.

3) <u>do-while loop</u>

The general form of the do-while loop is:-

```
do
{
  body of the loop
}while (test-condition);
```

The while loop makes a test of the condition <u>before</u> the loop is executed. So, the body of the loop may not be executed at all if the condition is not satisfied at the very first attempt. In some cases, it might be necessary to execute the body of the loop before the test is performed. Such situations can be handled with the <u>do-while loop</u>.

On reaching the do statement, the program evaluates the body of the loop first. At the end

Tanumoy Nag

Scanned by CamScanner

of the loop, the ~~test~~ test-condition in the while statement is evaluated. If the condition is true, the program continues to evaluate the body of the loop once again. This process continues as long as the condition is true.

Eg. to display all integers from 1 to 10, using do-while loop:-

```
i=1; // Initialization
do
{
    printf("%d",i);
    i++; // Increment
}while (i<=10);
```

↑ test-condition

Tanumoy Nag

Scanned by CamScanner

TN

Difference between while loop & do-while loop

while loop	do-while loop
1) It is entry-controlled loop.	1) It is exit-controlled loop.
2) The while loop tests the condition before executing any of the statements within the while loop.	2) The do-while loop tests the condition after having executed the statements within the loop.
3) At first, the condition is tested. If it is true, only then the ~~loop is~~ statements in the loop are executed. So, if the condition is found to be false the very first time, then the statements inside the loop are not executed even once.	3) At first, the statements inside the loop are executed once. Thereafter, the test condition is evaluated. So, if the condition is found to be false the very first time, even than the statements inside the loop are executed once.
4) The general form of while loop is as follows:-	4) The general form of do-while loop is as follows:-

```
initialise loop counter;
while (test condition is true)
{
    statement 1;
    statement 2;
    - - - - - -
    increment/decrement loop counter.
}
```

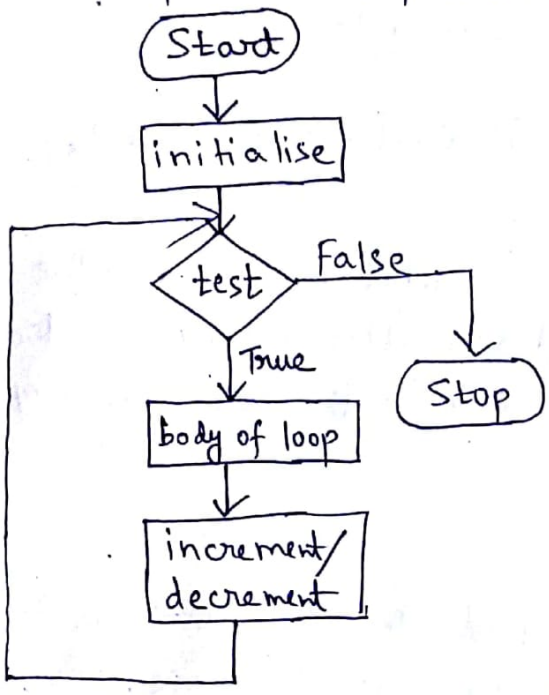
```
initialise loop counter;
do
{
    statement 1;
    statement 2;
    . . . . . .
    increment/decrement loop counter.
} while (test condition is true);
```

P.T.O

Tanumoy Nag

Scanned by CamScanner

5) The working of while loop can be represented as follows:-



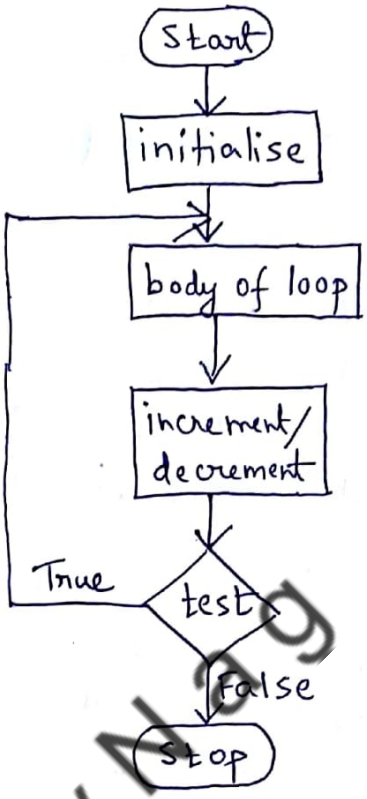
6) Example:-

Displaying all integers from 1 to 10 using while loop:-

```
int i=1; //Initialisation
while (i<=10)
{
    printf("%d", i);
    i++; //Increment
}
```

test condition

5) The working of do-while loop can be represented as follows:-



6) Example:-

Displaying all integers from 1 to 10 using do-while loop:-

```
int i=1; //Initialisation
do
{
    printf("%d", i);
    i++; //Increment
}while (i<=10);
```

test condition

Tanumoy Nag

Scanned by CamScanner